

Absolutely. We'll stop here for today and treat this as your **Day 4 Spring Boot/Spring Security session**. Tomorrow we'll continue from exactly where we stopped rather than restarting.

Spring Boot — Day 4 Complete Interview Notes

Date: 9 September 2026

Today you extended your existing **Spring Boot Staff Training Project** with **Spring Security, BCrypt authentication, and JWT fundamentals**.

Your project progression is now:

Client / Postman



Controller



Service



Repository



JPA / Hibernate



MySQL

with security added around the application:

Client



Spring Security



Authentication



JWT



Authorization



Controller

1. What is Spring Security?

Interview answer

Spring Security is a framework used to handle authentication and authorization in Spring applications. Authentication verifies the identity of the user, while authorization determines which resources and operations an authenticated user is allowed to access.

Simple memory

Authentication → WHO are you?

Authorization → WHAT can you access?

2. Authentication vs Authorization

Authentication

Authentication verifies the identity of the user.

Example:

Username + Password

↓

Verify credentials

↓

User authenticated 

Authorization

Authorization determines what the authenticated user is allowed to do.

Example:

Manager

├ View staff 

 lit staff ☐

└ Delete staff 

Staff

 ew own profile ☐

└ Delete staff 

Interview answer

“Authentication verifies the identity of a user, while authorization determines which resources and operations the authenticated user is allowed to access.”

3. Why shouldn't we store passwords as plain text?

Never store:

```
password = aniket123
```

Instead:

```
password = BCrypt hash
```

Interview answer

“We should not store passwords in plain text because if the database is compromised, attackers could directly obtain the passwords. Instead, passwords should be securely hashed before storing them.”

4. Hashing vs Encryption

Hashing

Hashing is generally a **one-way** process.

```
Password
```

↓

```
Hash
```

You don't decrypt the hash to get the original password.

Encryption

Encryption is designed to be reversible using a key.

```
Data
```

↓

Encryption
↓
Encrypted data
↓
Decryption
↓
Original data

Interview answer

“Hashing is a one-way process that converts data into a hash, while encryption is a reversible process where encrypted data can be decrypted using a key. Passwords should normally be stored using password hashing rather than encryption.”

5. What is BCrypt?

BCrypt is a **password-hashing algorithm**.

Password
↓
BCrypt
↓
Hash
↓
Database

Interview answer

“BCrypt is a password-hashing algorithm used to securely store passwords. In Spring Security, BCryptPasswordEncoder is an implementation of the PasswordEncoder interface that uses BCrypt.”

6. What is PasswordEncoder?

PasswordEncoder is a Spring Security interface used for password encoding/hashing and password verification.

You configured:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Here:

```
PasswordEncoder
  ↑
  |
BCryptPasswordEncoder
```

Interview answer

“PasswordEncoder is an interface provided by Spring Security for securely encoding passwords and verifying them. In our project, we use BCryptPasswordEncoder as its implementation.”

7. What is @Bean?

You learned that:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

tells Spring to create and manage the returned object in the Spring container.

Interview answer

“@Bean tells Spring to create and manage the returned object as a Spring bean so it can be injected and used by other components.”

8. What is a Salt?

A salt is a random value used during password hashing.

It means the same password can result in different hashes.

```
aniket123 + Salt A
  ↓
```

Hash A

aniket123 + Salt B

↓

Hash B

Interview answer

“A salt is a random value added during password hashing. It ensures that the same password can produce different hashes and makes attacks using precomputed hash tables more difficult.”

9. Why do we use `matches()`?

We learned that BCrypt can produce different hashes for the same password because of the salt.

Therefore, don't simply do:

```
newHash == storedHash
```

Instead:

```
passwordEncoder.matches(  
    enteredPassword,  
    storedHash  
);
```

Interview answer

“We use `matches()` because BCrypt uses a salt, so hashing the same password again can produce a different hash. `matches()` verifies the entered password against the stored BCrypt hash.”

10. User Entity

You created a separate User entity:

```
User  
-- id  
-- username
```

- password
- role

Why separate it from Staff?

Because they have different responsibilities.

Staff

↓

Business information

User

↓

Authentication information

Interview answer

“We keep authentication data separate from the Staff entity because they have different responsibilities. Staff represents business or employee information, while User represents authentication information such as username, password hash, and role.”

11. UserRepository

You created:

```
public interface UserRepository
    extends JpaRepository<User, Integer> {

    Optional<User> findByUsername(String username);
}
```

Why findByUsername()?

During login, Spring Security needs to find the user in the database.

username

↓

findByUsername()

↓

User

↓

password hash + role

Interview answer

“findByUsername() retrieves a user from the database using their username, which is required during authentication.”

12. UserDetailsService

This was an important concept today.

UserDetailsService connects Spring Security’s authentication mechanism with your database.

```
Spring Security
↓
UserDetailsService
↓
UserRepository
↓
MySQL
```

Interview answer

“UserDetailsService is used by Spring Security to load a user’s authentication information, usually by username. In our project, the custom implementation uses UserRepository to retrieve the user from the database.”

13. CustomUserDetailsService

You created:

```
@Service
public class CustomUserDetailsService
    implements UserDetailsService
```

It loads:

```
username
password hash
role
```

from your database and provides that information to Spring Security.

Important distinction

UserRepository



Database operations

UserDetailsService



Provides user information to Spring Security

14. AuthenticationManager

You learned that AuthenticationManager coordinates the authentication process.

Username + Password



AuthenticationManager



UserDetailsService



UserRepository



Database



PasswordEncoder



BCrypt verification

Interview answer

“AuthenticationManager is responsible for performing the authentication process. It verifies the user’s credentials and returns an authenticated result if the credentials are valid.”

15. Login Flow

This is one of the **most important interview answers from today.**

Complete flow

User enters username + password



Login request



Spring Boot



AuthenticationManager



UserDetailsService



UserRepository



User from DB



PasswordEncoder



BCrypt verification



Authentication successful



Generate JWT



Send JWT to client

Interview answer

“First, the user enters their username and password and sends a login request. Spring Security authenticates the user, and PasswordEncoder verifies the entered password against the stored BCrypt hash. If authentication is successful, the server generates a JWT and sends it to the client. The client then sends the JWT with subsequent requests to protected APIs.”

 You explained this correctly yourself today.

16. What is JWT?

JWT stands for **JSON Web Token**.

It is used to represent authenticated information and allow the client to authenticate subsequent requests.

Flow

```
Login successful
  ↓
JWT generated
  ↓
Client receives JWT
  ↓
Client sends JWT
  ↓
Protected API
```

Interview answer

“JWT is a JSON Web Token used to represent authentication information. After successful login, the server generates a JWT and sends it to the client, which then sends the token with subsequent requests to protected APIs.”

17. JWT Structure

A JWT contains three parts:

HEADER.PAYLOAD.SIGNATURE

Header

Contains information such as the signing algorithm.

Payload

Contains claims.

Your implementation uses:

`.subject(username)`

and:

`.issuedAt(...)`

`.expiration(...)`

So conceptually:

```
{  
  "sub": "aniket",  
  "iat": "...",  
  "exp": "..."  
}
```

Signature

Used to verify the token's authenticity and integrity.

18. JWT Signature

You correctly answered this today.

Interview answer

“The JWT signature is used to verify the authenticity and integrity of the token. It helps ensure that the JWT was signed by a trusted party and has not been modified.”

19. JWT Expiration

You learned why JWTs should expire.

If a token is valid forever, a stolen token could potentially be used indefinitely.

```
JWT  
↓  
Valid for limited time  
↓  
Expires  
↓  
Rejected ❌
```

Interview answer

“A JWT has an expiration time so that it remains valid only for a limited period. This improves security because a stolen or compromised token cannot be used indefinitely.”

20. Bearer Token

The client sends the JWT using the Authorization header:

Authorization: Bearer <JWT>

Bearer, not “barrier.” 😊

Interview answer

“Bearer is the authentication scheme used in the Authorization header. It tells the server that the following value is the authentication token.”

21. Access Token

An access token is used to access protected APIs.

Client

↓

Access Token

↓

Protected API

Usually it has a relatively short lifetime.

22. Refresh Token

A refresh token is different from an access token.

Access Token

↓

Expires

↓

Refresh Token

↓

New Access Token

Interview answer

“An access token is used to access protected APIs and usually has a short lifetime. A refresh token is used to obtain a new access token when the access token expires, so the user does not have to log in again.”

You learned the **concept** today, but we haven't implemented refresh tokens yet.

23. SecurityContext

After the JWT is validated, Spring Security needs to know:

Who is making this request?

The authentication information is placed into the SecurityContext.

Valid JWT

↓

Username

↓

Authentication object

↓

SecurityContext

↓

Spring Security knows user is authenticated

Interview answer

“The SecurityContext holds the authentication information for the current request, allowing Spring Security to know which user has been authenticated.”

24. JWT Filter

You started learning the JWT authentication filter today.

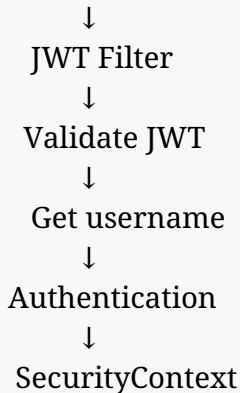
Its job is to:

1. Read Authorization header

2. Extract Bearer token
3. Extract username
4. Validate JWT
5. Create Authentication
6. Put authentication into SecurityContext

Flow:

Authorization: Bearer JWT



⚠ This is the one part we have started but not completely finished/testing yet.

25. Stateless Authentication

You learned:

SessionCreationPolicy.**STATELESS**

JWT authentication generally doesn't depend on a server-side login session.

Instead:

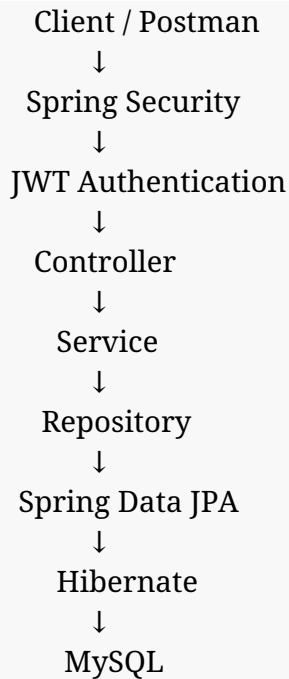
Request 1 → JWT
Request 2 → JWT
Request 3 → JWT

Interview answer

“With stateless JWT authentication, the server does not maintain a traditional authentication session. Each request contains the token needed to authenticate the user.”

26. Your Current Project Architecture

Your project has now evolved into:



And your project currently contains roughly:

```
com.example.springboot_training
|
-- config
|   └─ SecurityConfig.java
|
-- controller
|   └─ StaffController.java
|   └─ AuthController.java
|
-- dto
|   └─ LoginRequest.java
|
-- exception
|   └─ GlobalExceptionHandler.java
|
-- model
|   └─ Staff.java
```

```
|   └─ User.java
|
|-- repository
|   ├── StaffRepository.java
|   └─ UserRepository.java
|
|-- security
|   └─ JwtAuthenticationFilter.java
|
|-- service
|   ├── StaffService.java
|   ├── UserService.java
|   ├── CustomUserDetailsService.java
|   └─ JwtService.java
```

Interview Questions + Your Final Answers

These are the questions I asked you today and the polished answers you should revise.

Q1. What is Spring Security?

Spring Security is a framework used to handle authentication and authorization in Spring applications.

Q2. Authentication vs Authorization?

Authentication verifies the identity of the user, while authorization determines which resources and operations an authenticated user is allowed to access.

Q3. Why not store plain-text passwords?

We should not store passwords in plain text because if the database is compromised, attackers could directly obtain the passwords. We use password hashing instead.

Q4. What is BCrypt?

BCrypt is a password-hashing algorithm used to securely store passwords. BCryptPasswordEncoder is an implementation of Spring Security's PasswordEncoder interface.

Q5. Why use PasswordEncoder?

PasswordEncoder is used to securely hash passwords and verify passwords during authentication.

Q6. What is a salt?

A salt is a random value added during password hashing. It ensures that the same password can produce different hashes.

Q7. Why matches()?

We use matches() because BCrypt uses a salt, so hashing the same password again can produce a different hash. matches() verifies the entered password against the stored hash.

Q8. What is UserDetailsService?

UserDetailsService loads a user's authentication information for Spring Security, usually using the username.

Q9. What is AuthenticationManager?

AuthenticationManager performs the authentication process and verifies the user's credentials.

Q10. What is JWT?

JWT is a JSON Web Token used to represent authentication information. After successful login, the server generates the JWT and sends it to the client.

Q11. Why do we need JWT?

JWT allows the client to authenticate subsequent requests to protected APIs without sending the password with every request.

Q12. What are the three parts of JWT?

Header, Payload, and Signature.

Q13. What is the JWT signature?

The signature verifies the authenticity and integrity of the JWT and helps ensure that it has not been modified.

Q14. Why JWT expiration?

JWT expiration limits how long a token can be used and reduces the risk if the token is stolen.

Q15. What is Bearer?

Bearer is the authentication scheme used in the Authorization header to indicate that the following value is the authentication token.

Q16. What happens after successful login?

The server authenticates the user, generates a JWT, and sends it to the client. The client sends the JWT with subsequent requests to protected APIs.

Q17. Why SecurityContext?

It stores the authentication information for the current request so Spring Security knows which user is authenticated.



Day 4 Final Status

Spring Security fundamentals	✓
Authentication / Authorization	✓
Password hashing	✓
BCrypt	✓
Salt	✓
PasswordEncoder	✓
User entity	✓
UserRepository	✓
UserDetailsService	✓
AuthenticationManager	✓
Registration	✓
Login	✓
JWT concepts	✓
JWT generation	✓
JWT structure	✓
JWT signature	✓
JWT expiration	✓
Bearer token	✓
Access/Refresh token concepts	✓
JWT Filter	●

Protected Staff APIs
Role-based authorization



Stop point for today

Don't continue coding today. You've covered a lot of new Spring Security concepts, and you've already successfully generated a JWT and authenticated a real database user.

Tomorrow — Day 5

Tomorrow we continue **exactly from the unfinished portion**, not from the beginning.

 **Topic: Finish JWT Security + Role-Based Authorization**

Order:

1. Finish JwtAuthenticationFilter



2. Add filter to SecurityFilterChain



3. Test JWT with GET /api/staff



4. Test invalid/expired JWT



5. Understand SecurityContext properly



6. Add MANAGER / STAFF roles

↓

7. Configure authorization

↓

8. Protect Manager operations

↓

9. Test allowed/denied operations

↓

10. Complete interview revision

Then we'll have the important milestone:

Login

↓

BCrypt Authentication

↓

JWT

↓

JWT Filter

↓

SecurityContext

↓

Authentication

↓

Authorization

↓

MANAGER / STAFF

↓

Protected Staff API 

That will complete the **Spring Security + JWT milestone of your training project** before we move further into the next part of your one-month plan.

Tomorrow, just say: “Start Day 5” and we’ll continue from JwtAuthenticationFilter without repeating today’s material. 🔥